

Pontifícia Universidade Católica de Campinas
Escola Politécnica

ESTUDOS DE BANCO DE DADOS I

Prof Dra Renata Arantes
E-mail: renata.arantes@puc-campinas.edu.br

Objetivos da Aula

- Compreender o que são funções SQL e onde aplicá-las.
- Conhecer funções de texto, numéricas, datas, conversão e agregação.
- Aplicar funções nas tabelas (Cliente, Vendedor, Livro, Venda, Item_Venda).
- Praticar consultas reais do mini-mundo da Livraria BookStore.

Funções (Conceito)

- **O que é uma Função em Banco de Dados?**

Em bancos de dados relacionais, uma função é uma operação predefinida que realiza cálculos ou transforma dados e retorna um valor, seja ele único (como em uma função numérica ou de data) ou agregado (como em funções de contagem ou média).

- Rotinas internas do Oracle que operam sobre valores e retornam um resultado.
- Usadas no SELECT/INSERT/UPDATE/WHERE/GROUP BY/HAVING.
- Formatam, calculam e transformam dados durante a consulta.

Para Que Serve e Por Que Usar Funções?

Funções são úteis para otimizar a manipulação de dados, permitindo que as operações ocorram diretamente na consulta SQL, o que reduz o trabalho na aplicação e melhora a performance e a consistência dos resultados. Além disso, elas tornam o código mais limpo e fácil de manter.

Por exemplo:

- Agilidade:** Em vez de calcular o total de uma coluna em sua aplicação, você pode usar SUM diretamente na consulta SQL.
- Otimização de Processamento:** Bancos de dados são otimizados para lidar com grandes volumes de dados, então usar funções no banco é geralmente mais rápido do que processar esses dados no código de aplicação.

Onde se aplicam as funções

- SELECT: formatar/transformar colunas no resultado.
- INSERT: gerar/formatar valores no momento da inserção.
- UPDATE: recalcular e atualizar campos com base em funções.
- WHERE/GROUP BY/HAVING: filtrar, agrupar e agregar com funções.

Exemplos – SELECT / INSERT / UPDATE / WHERE / GROUP BY

/* SELECT: formatando dados */

```
SELECT INITCAP(nome) AS nome_formatado,  
       TO_CHAR(data_venda, 'dd/mm/yyyy') AS data_formatada  
FROM venda v JOIN cliente c ON c.cod_cliente = v.cod_cliente;
```

/* INSERT: usando função para gerar data atual */

```
INSERT INTO venda (cod_venda, data_venda, cod_cliente, cod_vendedor)  
VALUES (1001, SYSDATE, 1, 10);
```

/* UPDATE: reajustando preço com função numérica */

```
UPDATE livro  
SET preco = ROUND(preco * 1.05, 2)  
WHERE autor = 'Autor X';
```

/* WHERE: filtrar por mês/ano com TO_CHAR em data */

```
SELECT * FROM venda  
WHERE TO_CHAR(data_venda, 'YYYY-MM') = '2025-10';
```

/* GROUP BY/HAVING: agregando e filtrando por total */

```
SELECT cod_vendedor, SUM(preco_venda * qtde) AS total_vendido  
FROM item_venda  
GROUP BY cod_vendedor  
HAVING SUM(preco_venda * qtde) > 1000;
```

Classificação das funções SQL

3.1 Funções de Texto - Aplicações: padronização de nomes, extração de trechos, junção de campos.

Função	Descrição	Exemplo	Resultado
UPPER()	Converte para maiúsculo	UPPER('livro')	LIVRO
LOWER()	Converte para minúsculo	LOWER('LIVRO')	livro
INITCAP()	Primeira letra maiúscula	INITCAP('maria da silva')	Maria Da Silva
LENGTH()	Retorna o tamanho do texto	LENGTH('Livro A')	7
SUBSTR()	Retorna parte de uma string	SUBSTR('Cliente',1,3)	Cli
CONCAT()	Junta duas strings	CONCAT('Livro',' A')	Livro A

Usadas para manipular valores VARCHAR ou CHAR. /* Converte nomes para maiúsculas */
`SELECT UPPER(nome) AS nome_maiusculo
FROM cliente;`

💡 Aplicando na tabela da livraria:
`SELECT INITCAP(nome) AS Nome_Formatado FROM cliente;`

Funções de Texto – Exemplos (BookStore)

```
/* Padronizar nomes de clientes */
```

```
SELECT INITCAP(nome) AS nome_formatado  
FROM cliente;
```

```
/* Tamanho do título do livro */
```

```
SELECT titulo, LENGTH(titulo) AS tam_titulo  
FROM livro;
```

```
/* Prefixo dos 3 primeiros caracteres do autor */
```

```
SELECT autor, SUBSTR(autor, 1, 3) AS prefixo  
FROM livro;
```

```
/* Unir título e autor */
```

```
SELECT CONCAT(titulo, ' - ' || autor) AS titulo_autor  
FROM livro;
```

```
/* Trocar 'Edição' por 'Ed.' no título */
```

```
SELECT REPLACE(titulo, 'Edição', 'Ed.') AS titulo_curto  
FROM livro;
```

Classificação das funções SQL

3.2 Funções Numéricas

Aplicações: descontos, totais de venda, médias e auditoria.

Usadas para cálculos e arredondamentos.

Função	Descrição	Exemplo	Resultado
ROUND(n,dec)	Arredonda número	ROUND(45.678,2)	45.68
TRUNC(n,dec)	Trunca número	TRUNC(45.678,1)	45.6
MOD(x,y)	Retorna resto da divisão	MOD(10,3)	1
ABS(n)	Valor absoluto	ABS(-5)	5

💡 **Aplicando na livraria:**

```
SELECT titulo, preco, ROUND(preco*0.9,2) AS Preco_Com_Desconto FROM livro;
```

Numéricas – Exemplos práticos

/* Reajuste de preços */

```
SELECT titulo, preco, ROUND(preco * 0.90, 2) AS preco_promocional  
FROM livro;
```

/* Arredondar e truncar */

```
SELECT preco, ROUND(preco, 1) AS preco_round, TRUNC(preco, 1) AS  
preco_trunc  
FROM livro;
```

/* CEIL/FLOOR em cálculo de caixas (exemplo) */

```
SELECT CEIL(estoque/12) AS caixas_cheias, FLOOR(estoque/12) AS  
caixas_completas  
FROM livro;
```

Classificação das funções SQL

3.3 Funções de Data e Hora

Manipulam e formatam valores de data (DATE).

Função	Descrição	Exemplo	Resultado
<code>SYSDATE</code>	Data/hora atual do servidor	<code>SELECT SYSDATE FROM dual;</code>	12/10/2025
<code>ADD_MONTHS(data,n)</code>	Soma meses	<code>ADD_MONTHS(SYSDATE,6)</code>	+6 meses
<code>MONTHS_BETWEEN(d1,d2)</code>	Diferença em meses	<code>MONTHS_BETWEEN(SYSDATE,'01/01/2025')</code>	9.3
<code>NEXT_DAY(data,'TERÇA')</code>	Próxima terça-feira		
<code>LAST_DAY(data)</code>	Último dia do mês		

💡 **Aplicando na livraria:**

```
SELECT cod_venda, data_venda, ADD_MONTHS(data_venda,1) AS Garantia FROM venda;
```

Datas – Exemplos práticos (BookStore)

/* Formatar data da venda */

```
SELECT cod_venda, TO_CHAR(data_venda, 'dd/mm/yyyy') AS data  
FROM venda;
```

/* Próxima sexta após a venda */

```
SELECT cod_venda, NEXT_DAY(data_venda, 'SEXTA-FEIRA') AS prox_sexta  
FROM venda;
```

/* Garantia de 30 dias */

```
SELECT cod_venda, data_venda, data_venda + 30 AS fim_garantia  
FROM venda;
```

/* Meses decorridos até hoje */

```
SELECT cod_venda, ROUND(MONTHS_BETWEEN(SYSDATE, data_venda),2) AS meses  
FROM venda;
```

/* Último dia do mês da venda */

```
SELECT cod_venda, LAST_DAY(data_venda) AS ultimo_dia_mes  
FROM venda;
```

Classificação das funções SQL

3.4 Funções de Conversão

Usadas para mudar o tipo de dado (texto ↔ número ↔ data).

Função	Conversão	Exemplo
TO_CHAR(data, 'dd/mm/yyyy')	Data → texto	TO_CHAR(SYSDATE, 'dd/mm/yyyy')
TO_NUMBER('123')	Texto → número	123
TO_DATE('20/10/2025','dd/mm/yyyy')	Texto → data	20/10/2025

💡 Exemplo aplicado:

```
SELECT TO_CHAR(data_venda,'dd/mm/yyyy') AS Data_Formatada FROM venda;
```

Conversão – Exemplos práticos

/* Data → texto com máscara */

```
SELECT TO_CHAR(data_venda, 'dd/mm/yyyy HH24:MI') AS data_hora  
FROM venda;
```

/* Texto → data para filtro */

```
SELECT * FROM venda  
WHERE data_venda >= TO_DATE('01/10/2025', 'dd/mm/yyyy');
```

/* Número → texto com formatação */

```
SELECT titulo, TO_CHAR(preco, '999G990D00') AS preco_fmt  
FROM livro;
```

/* Texto → número e soma */

```
SELECT SUM(TO_NUMBER('100')) + SUM(preco) AS soma  
FROM livro;
```

Classificação das funções SQL

3.5 Funções de Agregação

Operam sobre conjuntos de linhas e retornam um valor agregado.

Função	Descrição	Exemplo
SUM()	Soma valores	SUM(preco_venda)
AVG()	Média	AVG(preco_venda)
MAX()	Maior valor	MAX(preco_venda)
MIN()	Menor valor	MIN(preco_venda)
COUNT()	Quantidade de registros	COUNT(*)

💡 **Aplicando na livraria:**

```
SELECT cod_vendedor, SUM(preco_venda*qtde) AS Total_Vendas  
FROM item_venda  
GROUP BY cod_vendedor;
```

Agregação – Exemplos (GROUP BY/HAVING)

/* Total vendido por vendedor */

```
SELECT v.cod_vendedor, SUM(iv.preco_venda * iv.qtde) AS total_vendido
FROM venda v
JOIN item_venda iv ON iv.cod_venda = v.cod_venda
GROUP BY v.cod_vendedor
ORDER BY total_vendido DESC;
```

/* Média de preços dos livros por autor */

```
SELECT autor, AVG(preco) AS preco_medio
FROM livro
GROUP BY autor
HAVING AVG(preco) > 50;
```

/* Contagem de vendas por mês */

```
SELECT TO_CHAR(v.data_venda, 'YYYY-MM') AS ano_mes, COUNT(*) AS qtd_vendas
FROM venda v
GROUP BY TO_CHAR(v.data_venda, 'YYYY-MM')
ORDER BY ano_mes;
```

Uso combinado de funções

As funções podem ser aninhadas (uma dentro da outra) e usadas com WHERE, GROUP BY e ORDER BY.

💡 Exemplo prático:

```
SELECT INITCAP(nome) AS Cliente,  
       TO_CHAR(data_venda,'dd/mm/yyyy') AS Data,  
       ROUND(SUM(preco_venda*qtde),2) AS Total  
FROM cliente  
JOIN venda USING(cod_cliente)  
JOIN item_venda USING(cod_venda)  
GROUP BY nome, data_venda  
ORDER BY Total DESC;
```

Resumindo: Essas funções podem ser usadas para:

1.Realizar Cálculos: Funções como SUM, AVG, COUNT, etc., permitem calcular diretamente nas consultas, em vez de fazer o processamento na aplicação.

2.Manipular Dados: Funções de manipulação de texto, como UPPER ou REPLACE, permitem ajustar dados diretamente no banco (por exemplo, transformando nomes em maiúsculas para padronização).

3.Trabalhar com Datas: Funções como SYSDATE, ADD_MONTHS, e EXTRACT permitem realizar operações e cálculos com datas, como somar meses ou extrair partes específicas da data (dia, mês, ano).

4.Converter Dados: Funções de conversão (TO_CHAR, TO_DATE, TO_NUMBER) permitem converter valores de um tipo para outro, como transformar uma data em texto ou vice-versa, útil para formatações e comparações

Onde e Como Criar Funções Personalizadas

No SQL Developer (diretamente no banco de dados):

Você pode criar funções personalizadas usando comandos PL/SQL no SQL Developer.

As funções definidas pelo usuário ficam armazenadas no banco de dados e podem ser chamadas em qualquer consulta, como as funções nativas.

FUNÇÕES - SINTAXE

```
[-] CREATE FUNCTION function_name  
      (parameter)  
      RETURN return_datatype  
      IS  
          declaration_section  
      BEGIN  
          ...  
      END function_name;
```

Como Criar uma Função no SQL Developer

No SQL Developer, você pode definir uma função customizada com PL/SQL. Aqui está o passo a passo:

- Abra o SQL Developer e conecte-se ao banco de dados.
- Acesse para escrever o código da função.

Exemplo de criação de uma função que calcula um bônus com base no salário:

```
CREATE OR REPLACE FUNCTION calcular_bonus (p_salario NUMBER) - Criar ou atualizar uma função - parâmetro de entrada do tipo
RETURN NUMBER - Indica que a função retorna um valor numérico
IS
    v_bonus NUMBER; - v-bonus é a variável que armazena o cálculo do bônus

BEGIN
    v_bonus := p_salario * 0.10; -- Calcula 10% de bônus
    RETURN v_bonus;
END;
```

Usar a Função Criada em uma Consulta:

Após criar a função, você pode chamá-la em consultas SQL, como no exemplo abaixo:

```
SELECT nome, salario, calcular_bonus(salario) as bonus  
FROM empregados;
```

Aqui, a função `calcular_bonus` é aplicada em cada registro da coluna `salario` na tabela `empregados`

Criar funções no banco de dados via SQL Developer é uma excelente prática quando a lógica pode ser implementada diretamente em SQL, enquanto funções no backend são úteis quando a lógica exige integração com outras partes da aplicação.

Quando você aplica uma função em uma consulta SQL, o valor calculado é gerado e exibido apenas como resultado daquela consulta específica, mas **não é salvo permanentemente** na tabela. Para armazenar o novo valor, é necessário executar uma operação adicional que insira ou atualize os dados na tabela.

Como Salvar o Resultado de uma Função

Para salvar o valor retornado por uma função em uma coluna, você pode utilizar o comando UPDATE para modificar os dados existentes ou o INSERT para inserir novos registros, conforme a necessidade.

1. Atualizando uma Coluna Existente

Se quiser armazenar o valor retornado pela função em uma coluna de uma tabela, você pode fazer uma atualização direta. Por exemplo, vamos supor que a coluna bonus na tabela empregados vai armazenar o valor calculado:

UPDATE empregados

SET bonus = calcular_bonus(salario);

Nesse exemplo:

- A função `calcular_bonus(salario)` é aplicada a cada linha da coluna `salario`, e o resultado é salvo na coluna `bonus`.
- O comando `UPDATE` garante que o valor seja salvo permanentemente.

2. Inserindo o Valor em uma Nova Tabela

Se o objetivo for salvar o valor resultante em uma nova tabela, você pode usar um `INSERT INTO` com uma consulta `SELECT`. Por exemplo:

```
INSERT INTO empregados_com_bonus (nome, salario, bonus)  
SELECT nome, salario, calcular_bonus(salario) AS bonus  
FROM empregados;
```

Nesse exemplo:

- A função `calcular_bonus(salario)` calcula o bônus para cada registro da tabela `empregados`.
- O comando `INSERT INTO` insere esses valores na nova tabela `empregados_com_bonus`.

Funções Sintaxe

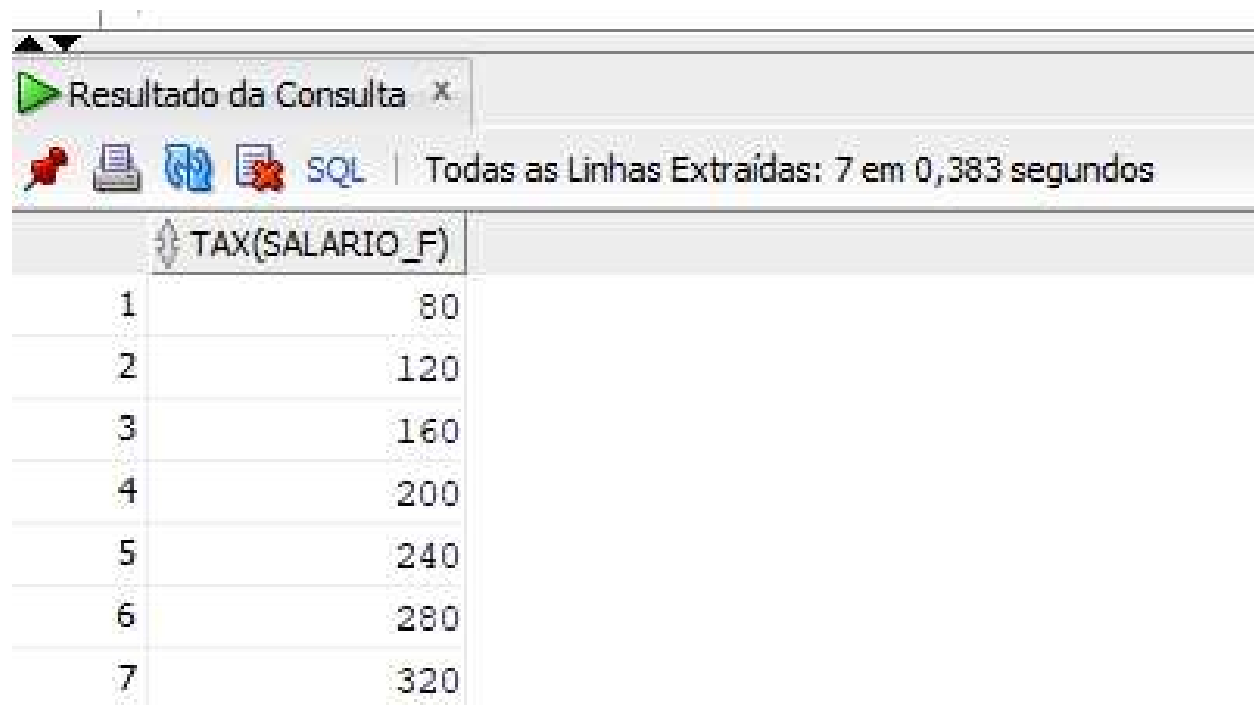
```
//Criando função:
```

```
CREATE FUNCTION Tax  
    (valor in number)  
RETURN number  
IS  
    BEGIN  
        return (valor*0.08);  
END Tax;
```

```
Invocando uma função:
```

```
select Tax(salario_f) from funcionario;  
  
drop function Tax;
```

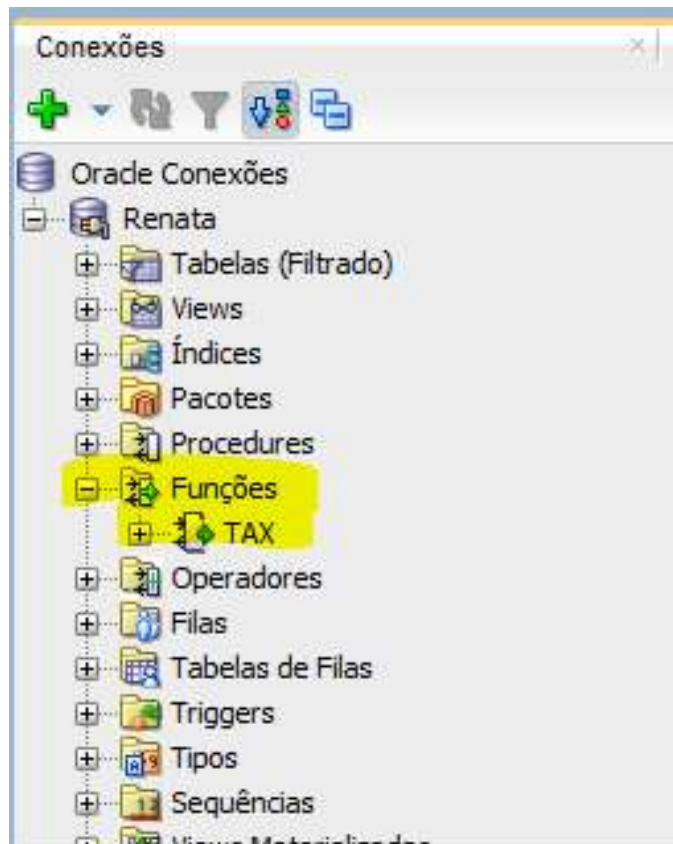
Resultado da Execução



Resultado da Consulta	
Todas as Linhas Extraídas: 7 em 0,383 segundos	
TAX(SALARIO_F)	
1	80
2	120
3	160
4	200
5	240
6	280
7	320

Funções

Apagar a Função



```
drop function Tax;
```

Funções

```
select CPF_FUNC , NOME_FUNC, Tax(salario_f) from funcionario;
```

Resultado da Consulta x			
SQL Todas as Linhas Extraídas: 7 em 0,037 segundos			
	CPF_FUNC	NOME_FUNC	TAX(SALARIO_F)
1	13	Daniela	80
2	2	Rodrigo	120
3	4	Rafael	160
4	6	Renata	200
5	8	Leticia	240
6	10	Andre	280
7	12	Vinicius	320

Funções

```
select CPF_FUNC , NOME_FUNC, Tax(salario_f) from funcionario  
where cpf_func= 10;
```

Resultado da Consulta ✕			
    SQL Todas as Linhas Extraídas: 1 em 0,016 segundos			
	CPF_FUNC	NOME_FUNC	TAX(SALARIO_F)
1	10	Andre	280

Funções

// Criando a função

```
CREATE FUNCTION fn_calculo  
    (a in number, b in number)  
    RETURN number  
    IS  
    BEGIN  
        return (a*b);  
    END fn_calculo;
```

Function FN_CALCULO compilado

Invocando uma função:

```
select nome_produto, fn_calculo(preco_produto,4) from produto;
```

Funções - Resultado

```
select nome_produto, fn_calculo(preco_produto,4) from produto;
```

Saída do Script x Resultado da Consulta x

    SQL | Todas as Linhas Extraídas: 4 em 0,003 segundos

	NOME_PRODUTO	FN_CALCULO(PRECO_PRODUTO,4)
1	S10 - CELULAR	14000
2	MAC PRO - COMPUTADOR	32000
3	OLED - TV	11600
4	APPLE WATCH	15920

Funções - Resultado

```
select nome_produto, fn_calculo(preco_produto,4) from produto  
where id_produto = 11;
```

Saída do Script x Resultado da Consulta x	
SQL Todas as Linhas Extraídas: 1 em 0,005 segundos	
NOME_PRODUTO	FN_CALCULO(PRECO_PRODUTO,4)
1 S10 - CELULAR	14000

EXEMPLO

- Crie uma função para aumentar 10% do valor em todos os produtos da tabela Produto

Tbl_Produto		
Cod_Produto	Descrição_Produto	Valor_Produto
1	Galaxy S22 Sansung	4.200,00
2	Smart Tv Led 50 4k	2.100,00
3	MacBook Pro	10.000,00
4	JBL BoomBox 2	2.500,00

EXEMPLO

```
CREATE FUNCTION AUMENTA_PRECO
(PRECO in NUMBER, TAXA in NUMBER)
RETURN number
IS
BEGIN
    return (PRECO +PRECO * TAXA/100);
END AUMENTA_PRECO;
```

Function AUMENTA_PRECO compilado

EXEMPLO

Invocando uma função:

```
select nome_produto, AUMENTA_PRECO(preco_produto,10) from produto;
```

Saída do Script x Resultado da Consulta x	
SQL Todas as Linhas Extraídas: 4 em 0,006 segundos	
NOME_PRODUTO	AUMENTA_PRECO(PRECO_PRODUTO,10)
1 S10 - CELULAR	3850
2 MAC PRO - COMPUTADOR	8800
3 OLED - TV	3190
4 APPLE WATCH	4378